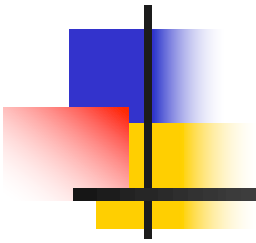
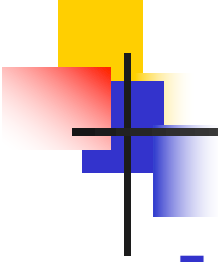


7.1 Arquitectura global

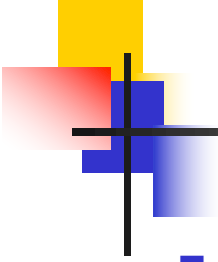


- Distribución fuente
- Desarrollo multi-modular
- Reductor raíz
- Bootstrap y React Bootstrap
- Layout y enrutamiento
- Internacionalización
- Capa Acceso a Servicios



Distribución fuente

- Visualizar <https://github.com/udc-fic-pa/pa-shop/tree/master/frontend>
- El frontend de pa-shop se creó con Vite, usando la plantilla para React
- Debajo de la carpeta **frontend** reside la estructura típica creada por Vite
 - `index.html`, `public`, `src`, `package.json`, etc.
- Debajo de `src`, entre otros
 - `backend`: capa Acceso a Servicios
 - `i18n`: internacionalización y ficheros de mensajes
 - `modules`: capa IU
 - `store`: creación del `store`
 - `main.jsx`

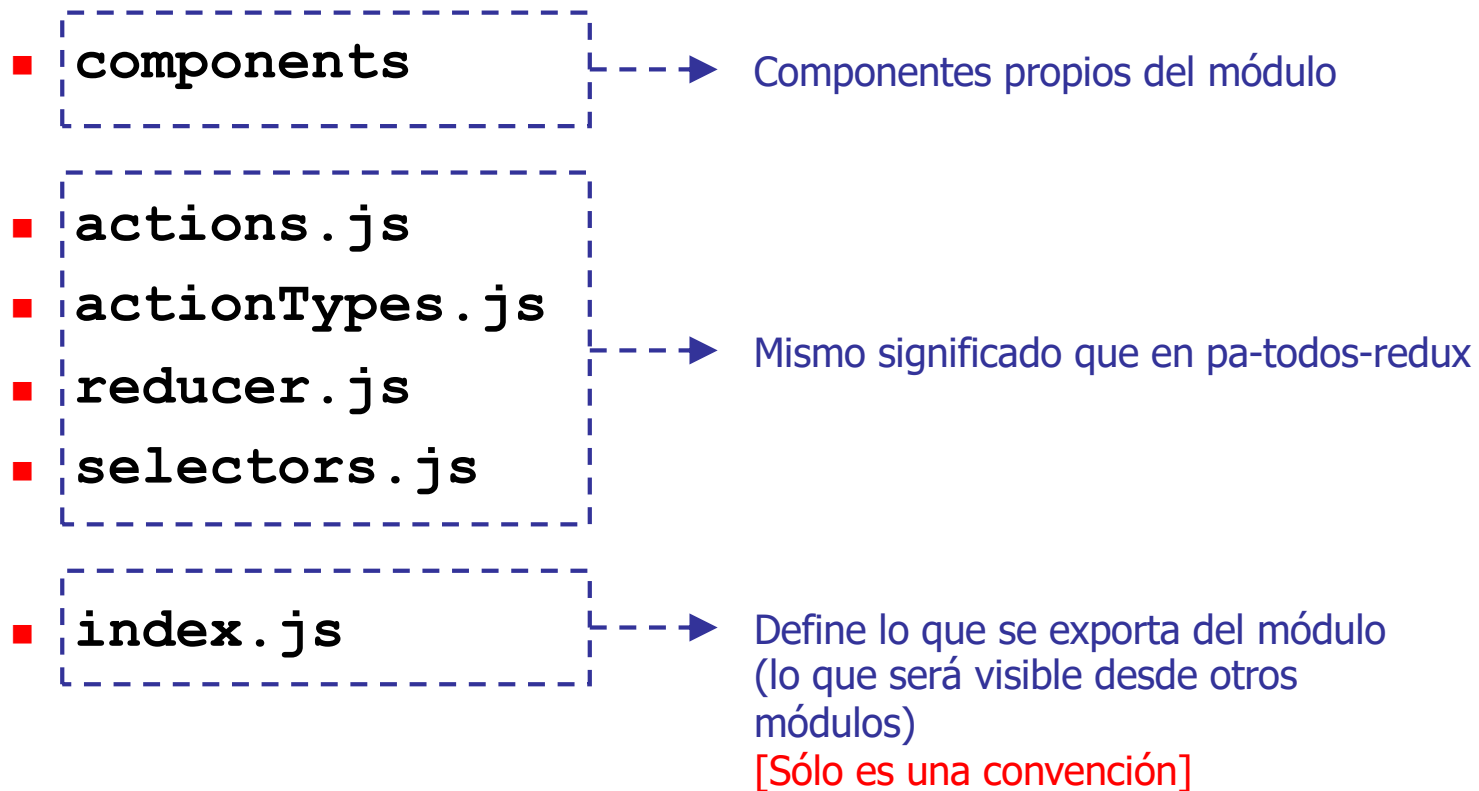


Desarrollo multi-modular

- Para facilitar la gestión del código, la capa IU está organizada en “módulos” (**modules**)
- Entre otros, **modules** contiene
 - **users, catalog, shopping**
 - Cada uno de estos módulos contiene el código de la IU correspondiente a los casos de uso expuestos por los controladores **UserController**, **CatalogController** y **ShoppingController**
 - No hay porque hacerlo así
 - Es sólo una forma de agrupar casos de uso relacionados
- Además, contiene otros dos módulos
 - **app**: layout de la IU
 - **common**: componentes reusables entre módulos

Estructura de un módulo

- Cada módulo conceptualmente es una mini IU
- En general, cada módulo contiene



Estructura de un módulo – Estilo de exportación

- El fichero `index.js` de un módulo define lo que éste exporta
- Ejemplo: `catalog/index.js`

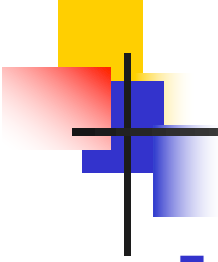
```
import * as actions from './actions';  
import * as actionTypes from './actionTypes';  
import reducer from './reducer';  
import * as selectors from './selectors';
```

Se exportan los componentes
que se usan desde otros módulos

```
export {default as FindProducts} from './components/FindProducts';  
export {default as FindProductsResult} from './components/FindProductsResult';  
export {default as ProductDetails} from './components/ProductDetails';
```

```
export default {actions, actionTypes, reducer, selectors};
```

Por sencillez se exportan todas las acciones, reductor y selectores



Estructura de un módulo – Estilo de importación

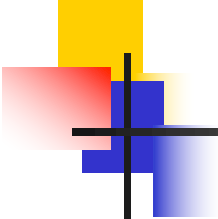
- Desde fuera del módulo

```
import {FindProductsResult, ProductDetails} from '<<...>>/catalog';
```

```
import catalog from '<<...>>/catalog';
```



Luego es posible emplear `catalog.reducer`,
`catalog.selectors.getCategories(state)`, etc.

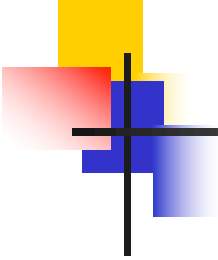


Estructura de un módulo – Ejemplo: catalog/reducer.js

```
import {combineReducers} from 'redux';
...
const initialState = {
  categories: null,
  productSearch: null
};

const categories = (state = initialState.categories, action) => {
  ...
}

const productSearch = (state = initialState.productSearch, action) => {
  ...
}
```

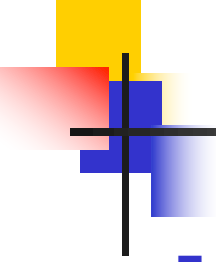
Estructura de un módulo – Ejemplo: catalog/reducer.js

```
const reducer = combineReducers({  
  categories,  
  productSearch  
});
```

```
export default reducer;
```

Produce un estado con una estructura del tipo

```
{  
  categories: ...,  
  productSearch: ...  
}
```



Reductor raíz

- Existe un reductor raíz que combina los reductores de los módulos
 - `store/rootReducer.js`

```
import {combineReducers} from 'redux';
```

```
import app from '../modules/app';
```

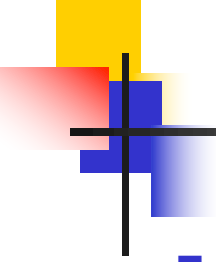
```
import users from '../modules/users';
```

```
import catalog from '../modules/catalog';
```

```
import shopping from '../modules/shopping';
```

```
const rootReducer = combineReducers({  
  app: app.reducer,  
  users: users.reducer,  
  catalog: catalog.reducer,  
  shopping: shopping.reducer  
});
```

```
export default rootReducer;
```



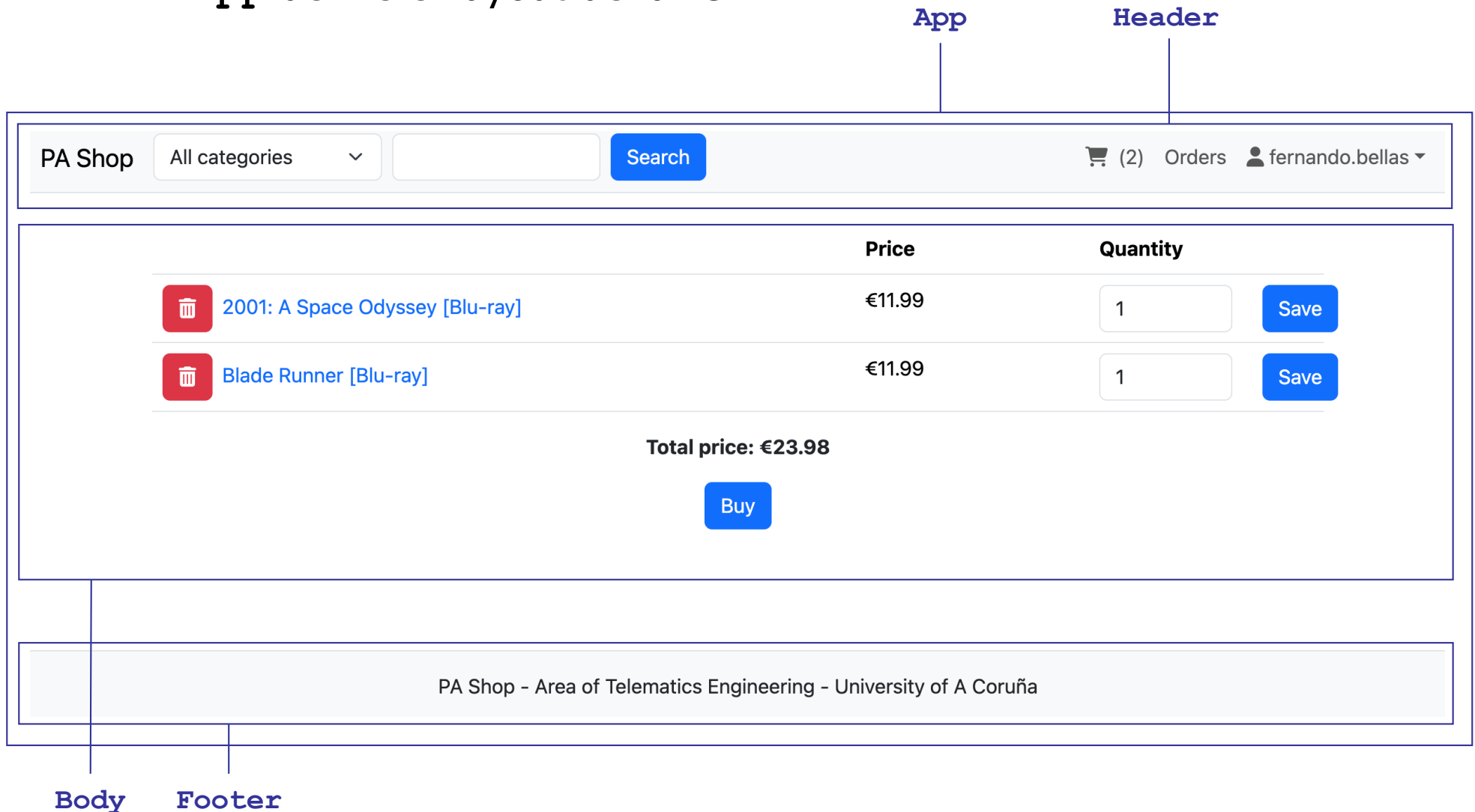
Reducer raíz

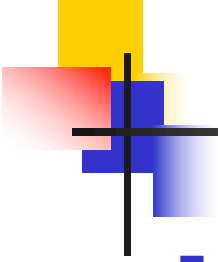
- Y produce un estado con una estructura del tipo

```
{  
  app: {  
    ...  
  },  
  users: {  
    ...  
  },  
  catalog: {  
    categories: ...,  
    productSearch: ...  
  },  
  shopping: {  
    ...  
  }  
}
```

Layout y enrutamiento

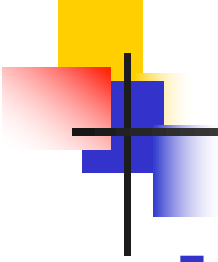
- `src/main.jsx` renderiza el componente **App** (módulo **app**)
- **App** define el layout de la IU





Bootstrap

- pa-shop hace uso de **Bootstrap** para estilizar la interfaz de usuario
 - <https://getbootstrap.com/>
- Es **responsive**
 - La interfaz de usuario se adapta al tamaño de la pantalla
- Internamente consta de dos partes
 - Una parte CSS
 - Y otra JavaScript
- La parte JavaScript modifica el DOM del navegador directamente, por lo que no es compatible con frameworks como React, Vue o Angular, que asumen un control total del DOM del navegador



React Bootstrap

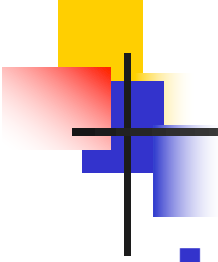
- Por este motivo, pa-shop usa la librería react-bootstrap
 - <https://react-bootstrap.github.io/>
- Requiere usar la parte CSS de Bootstrap
 - Pero aporta componentes React que abstraen al desarrollador de la mayor parte de estilos CSS de Bootstrap, aunque no de todos
- No depende de la parte JavaScript de Bootstrap
 - Las funcionalidades de Bootstrap que manipulan directamente el árbol DOM del navegador están reimplementadas desde cero mediante componentes React

Layout y enrutamiento – App.jsx

```
...  
const App = () => {  
  ...  
  return (  
    <div className="d-flex flex-column min-vh-100">  
      <Header/>  
      <Body/>  
      <Footer/>  
    </div>  
  );  
}  
  
export default App;
```

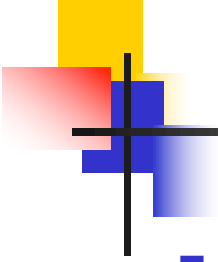
Estilos de Bootstrap

Definidos también en el módulo app



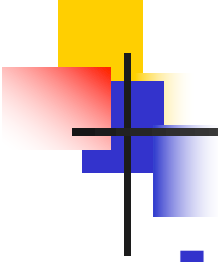
Layout y enrutamiento

- Para poder navegar entre las “pantallas” de la aplicación, el frontend usa una librería de enrutamiento
- Una librería de enrutamiento
 - Permite navegar entre “pantallas” mediante enlaces y provocar programáticamente un cambio de pantalla
 - Cambia la URL que muestra el navegador para que sea consistente con la pantalla que muestra la aplicación
- Una librería popular de enrutamiento para React es React Router
 - <https://reactrouter.com>



Layout y enrutamiento

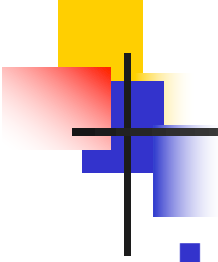
- Para poder usar React Router desde cualquier componente del frontend, debe usarse el componente **BrowserRouter**
 - Típicamente envuelve a **App**
 - Cumple una función similar al componente **Provider** de Redux
 - En este caso, proporciona información de enrutamiento a todos los componentes del árbol de componentes que contiene



Layout y enrutamiento

- En `src/main.jsx`

```
...  
import {BrowserRouter} from 'react-router';  
...  
  
/* Render application. */  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <Provider store={store}>  
      <IntlProvider locale={locale} messages={messages}>  
        <BrowserRouter>  
          <App/>  
        </BrowserRouter>  
      </IntlProvider>  
    </Provider>  
  </React.StrictMode>);
```



Layout y enrutamiento

- El componente `Link` permite insertar un enlace a una pantalla
- Ejemplo: `Header`

...

```
import {Link} from 'react-router';
```

...

```
const Header = () => (
```

```
    const userName = useSelector(users.selectors.getUserName);
```

...

```
<Link className="nav-link" to="/shopping/find-orders">
```

```
    ...Texto del enlace ...
```

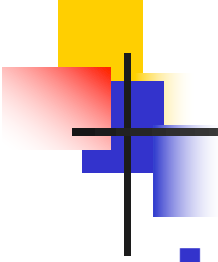
```
</Link>
```

...

NOTA: sólo se muestra si `userName !== null`

```
);
```

```
export default Header;
```



Layout y enrutamiento

- Aunque el fragmento de código anterior es totalmente correcto, en el código fuente de **Header** en realidad figura

...

```
import {Link} from 'react-router';  
import Nav from 'react-bootstrap/Nav';
```

...

```
<Nav.Link as={Link} to="/shopping/find-orders">  
    ...Texto del enlace ...  
</Nav.Link>
```

...

NOTA: sólo se muestra si `userName !== null`

- **Nav.Link** es un componente de React Bootstrap, que en este caso, genera un enlace con el componente **Link** de React Router, añadiendo el estilo **nav-link** (para enlaces que figuran en la cabecera)
- Se ha preferido usar esta segunda opción para ser consistentes con el hecho de usar la librería React Bootstrap

Layout y enrutamiento – Body.jsx

...

```
import {Route, Routes} from 'react-router';
```

...

```
const Body = () => (
```

```
  const loggedIn = useSelector(users.selectors.isLoggedIn);
```

```
  return (
```

Componente de React Bootstrap que genera un `<div>` al que le añade el estilo `container`

```
    <Container className="my-4 justify-content-center flex-grow-1">
```

...

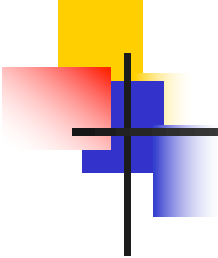
```
    <Routes>
```

Cada vez que se cambia de pantalla, `Routes` renderiza el `Route` cuyo `path` concuerde mejor (el más específico)

```
      <Route path="/" element={<Home/>}/>
```

```
      <Route path="/catalog/find-products-result"
        element={<FindProductsResult/>}/>
```

```
      <Route path="/catalog/product-details/:id"
        element={<ProductDetails/>}/>
```



Layout y enrutamiento – Body.jsx

Idioma:
conditional
rendering

```
...  
{!loggedIn && <Route path="/shopping/find-orders"  
  element={<FindOrders/>} />}
```

...

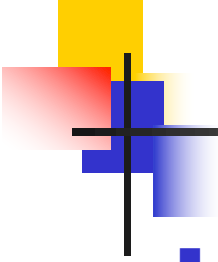
```
{!loggedIn && <Route path="/users/signup"  
  element={<SignUp/>} />}
```

```
</Routes>  
</Container>
```

```
);
```

```
}
```

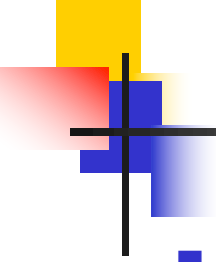
```
export default Body;
```



Layout y enrutamiento – cambios de pantalla

- Cada vez que se cambia de pantalla, bien porque se hizo clic en un enlace generado por **Link** o porque se provocó un cambio de pantalla programáticamente, se renderiza el **Route** cuyo **path** concuerde mejor (el que tenga una concordancia más específica), y con él, el componente especificado en **element**
 - Si se solicita un cambio de pantalla a `/catalog/find-products-result`, existen dos **Route** cuyo **path** concuerda: el **Route** de **Home** y el **Route** de **FindProductsResult**
 - **Routes** elige el **Route** de **FindProductsResult** porque concuerda mejor (en este caso, exactamente)
 - En pa-shop, el **path** en el **Route** de **Home** se especificó como `/*`, para que se renderice **Home** cuando se solicita un cambio de pantalla a `/` o a un path para el que no hay un **Route** cuyo **path** concuerde
 - Profesionalmente, se podría hacer algo de este estilo

```
<Route path="/" element={<Home/>}/>
<Route path="/*" element={<PageNotFound/>}/>
```



Layout y enrutamiento – parámetros

- Los paths especificados en **Route** pueden incluir partes variables, llamadas informalmente “parámetros”
- Como ejemplo, consideremos un enlace al detalle de un producto
- En **Body**

```
<Route path="/catalog/product-details/:id"  
      element={<ProductDetails/>} />
```


Layout y enrutamiento – parámetros

- Si se hace clic en el enlace generado por ...

```
<Link to="/catalog/product-details/1">  
  2001: A Space Odyssey [Blu-ray]  
</Link>
```

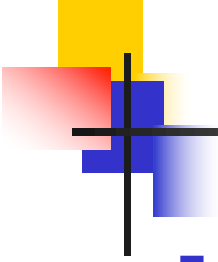
- ... se renderizará **ProductDetails**
- Desde **ProductDetails** se puede acceder a los parámetros del path mediante el Hook **useParams**

```
const {id} = useParams() ;
```

↓

"1"

Devuelve un objeto con una propiedad por cada "parámetro"



Internacionalización

- En el caso de estudio se utiliza React Intl como librería de internacionalización
 - <https://formatjs.github.io/docs/react-intl/>
- Entre otros, proporciona componentes para internacionalizar mensajes, cantidades numéricas y fechas
 - **FormattedMessage**
 - **FormattedNumber**
 - **FormattedDate**
 - **FormattedTime**

Internacionalización – inicialización

■ En src/main.jsx

...

```
import {IntlProvider} from 'react-intl';
```

...

```
import {initReactIntl} from './i18n';
```

...

```
/* Configure i18n. */
```

```
const {locale, messages} = initReactIntl();
```

Función proporcionada por el caso de estudio: devuelve el locale del navegador ('en' por defecto) y un objeto con los mensajes correspondientes a ese locale

```
/* Render application. */
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(
```

```
  <React.StrictMode>
```

```
    <Provider store={store}>
```

```
      <IntlProvider locale={locale} messages={messages}>
```

```
        <BrowserRouter>
```

```
          <App/>
```

```
        <BrowserRouter>
```

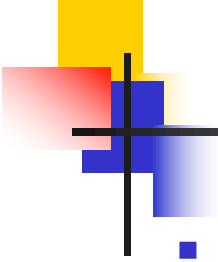
```
          </IntlProvider>
```

```
        </Provider>
```

```
      </React.StrictMode>);
```

IntlProvider cumple una función similar a Provider/BrowserRouter: inyecta el locale y los mensajes a los componentes hijo

...



Internacionalización – mensajes

- `src/i18n/messages`: contiene los ficheros de mensajes
- `messages_en.js`

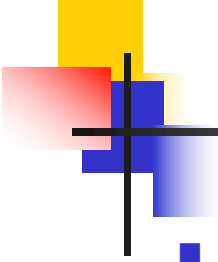
```
export default {  
  ...  
  'project.app.Header.logout': 'Logout',  
  ...  
}
```

- `messages_es.js`

```
...  
'project.app.Header.logout': 'Salir',  
...
```

- `messages_gl.js`

```
...  
'project.app.Header.logout': 'Saír',  
...
```



Internacionalización

- Ejemplo 1. En `ProductDetails.jsx` (módulo `catalog`)

```
<FormattedMessage id='project.global.fields.price' />{'': ' '}  
<FormattedNumber value={product.price}  
  style="currency" currency="EUR" />
```

Para el caso de cantidades monetarias

locale=en → Price: €11.99

locale=es → Precio: 11,99 €

locale=gl → Prezo: 11,99 €

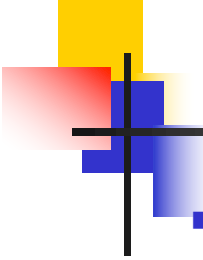
- Ejemplo 2. En `Orders.jsx` (módulo `shopping`)

```
<FormattedDate value={new Date(order.date)} /> - <FormattedTime  
value={new Date(order.date)} />
```

locale=en → 3/28/2019 - 7:40 PM

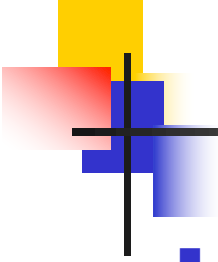
locale=es → 28/3/2019 - 19:40

locale=gl → 28/3/2019 - 19:40



Capa Acceso a Servicios – peticiones HTTP

- Para el envío de peticiones HTTP, el caso de estudio proporciona la función `appFetch` (`src/backend/appFetch.js`)
- Firma
 - `const appFetch = async (method, path, body)`
- Parámetros
 - [obligatorio] `method`: operación HTTP (e.g. 'GET', 'POST', 'PUT', etc.)
 - [obligatorio] `path`: path del recurso sobre el que se invoca la operación
 - La URL final es `<<backend URL>>/path`
 - `<<backend URL>>` → `frontend/.env.{development, production}`
 - <https://vitejs.dev/guide/env-and-mode.html>
 - [opcional] `body`: objeto cuya representación JSON se incluirá en el cuerpo
- Valor de retorno
 - Objeto con dos propiedades: `ok` y `payload`
 - `ok`: booleano
 - `true` si el código de respuesta HTTP es 2xx.
 - `payload`: objeto
 - Objeto construido a partir de la representación JSON que llegó en el cuerpo de la respuesta
 - `null` si la respuesta no contenía una representación JSON en el cuerpo

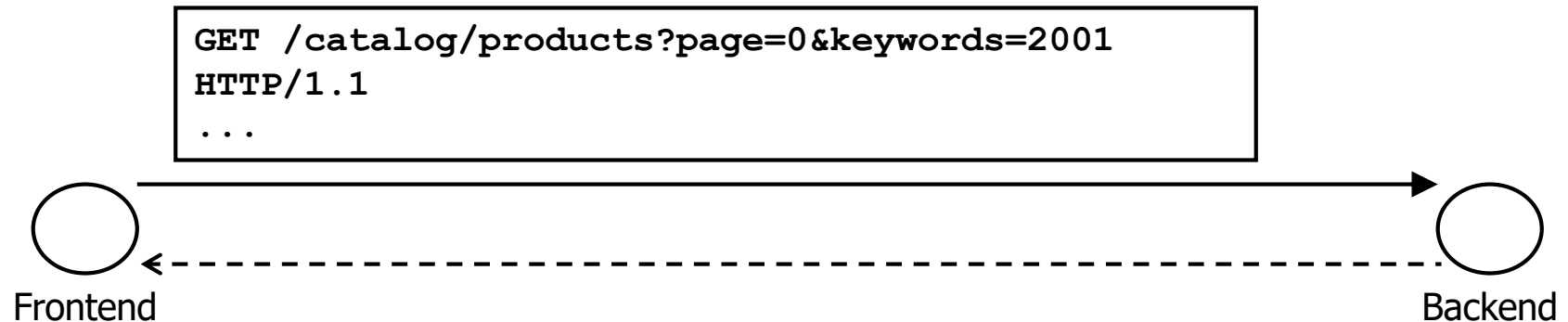


Capa Acceso a Servicios – peticiones HTTP

- Si no se puede enviar la petición o recibir la respuesta, **appFetch** causa indirectamente la apertura de una ventana modal de error
- **appFetch** también se encarga de enviar automáticamente el token JWT si el usuario se había autenticado previamente
- A nivel de implementación, **appFetch** usa la API estándar Fetch
 - Soportada nativamente por los navegadores modernos
 - https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 1



```
HTTP/1.1 200 OK
...
{
  "items": [
    {
      "id": 1,
      "name": "2001: A Space Odyssey [Blu-ray]",
      "categoryId": 1
    }
  ],
  "existMoreItems": false
}
```


Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 1

NOTA: este fragmento de código tiene que estar dentro de una función **async**

```
const response = await appFetch('GET',  
  '/catalog/products?page=0&keywords=2001');
```

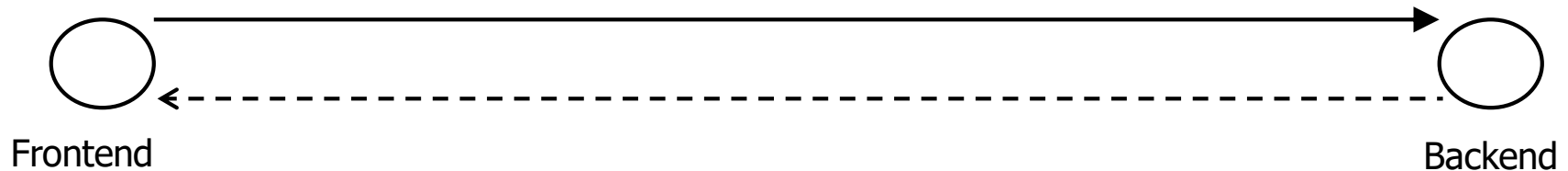
```
if (response.ok) {  
  doSomethingWithResult(response.payload);  
}
```

```
{  
  items: [  
    {  
      id: 1,  
      name: "2001: A Space Odyssey [Blu-ray]",  
      categoryId: 1  
    }  
  ],  
  existMoreItems: false  
}
```

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 2

```
POST /shopping/shoppingcarts/1/buy HTTP/1.1
...
{
  "postalAddress": "Rue del Percebe, 13",
  "postalCode": "12345"
}
```



```
HTTP/1.1 200 OK
...
12
```

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 2

NOTA: este fragmento de código tiene que estar dentro de una función **async**

```
const response = await appFetch('POST',  
    '/shopping/shoppingcarts/1/buy',  
    {postalAddress: "Rue del Percebe, 13", postalCode: "12345"});  
  
if (response.ok) {  
    doSomethingWithResult(response.payload);  
} else {  
    doSomethingWithErrors(response.payload);  
}
```

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 3

```
POST /shopping/shoppingcarts/1/buy
```

```
...
```

```
Accept-Language: en
```

La envía automáticamente `fetch` en todas las peticiones HTTP

```
Authorization: Bearer ...
```

La envía automáticamente `appFetch` una vez el usuario está autenticado

```
...
```

```
{
```

```
  "postalAddress": "Rue del Percebe, 13",
```

```
  "postalCode": "12345"
```

```
}
```



```
HTTP/1.1 404 Not Found
```

```
...
```

```
{
```

```
  "globalError": "shopping cart is empty"
```

```
}
```

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 3

NOTA: este fragmento de código tiene que estar dentro de una función **async**

```
const response = await appFetch('POST',  
    '/shopping/shoppingcarts/1/buy',  
    {postalAddress: "Rue del Percebe, 13", postalCode: "12345"});
```

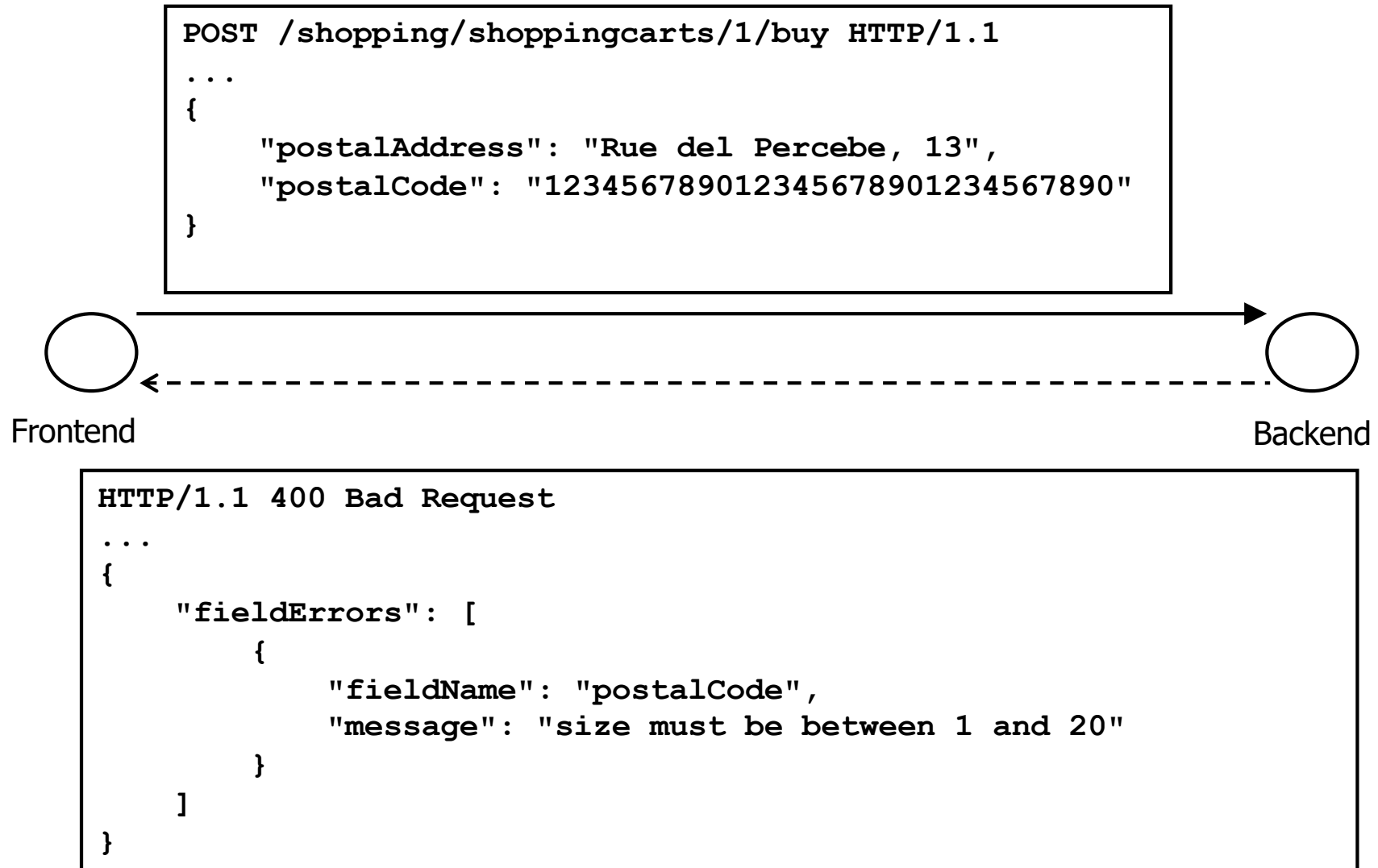
```
if (response.ok) {  
    doSomethingWithResult(response.payload);  
} else {  
    doSomethingWithErrors(response.payload);  
}
```

↓

```
{  
  globalError: "shopping cart is empty"  
}
```

Capa Acceso a Servicios – peticiones HTTP

■ Ejemplo 4



Capa Acceso a Servicios – peticiones HTTP

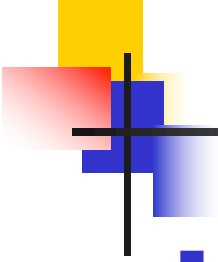
■ Ejemplo 4

NOTA: este fragmento de código tiene que estar dentro de una función **async**

```
const response = await appFetch('POST',  
  '/shopping/shoppingcarts/1/buy',  
  {postalAddress: "Rue del Percebe, 13", postalCode: "12345678..."});  
  
if (response.ok) {  
  doSomethingWithResult(response.payload);  
} else {  
  doSomethingWithErrors(response.payload);  
}
```

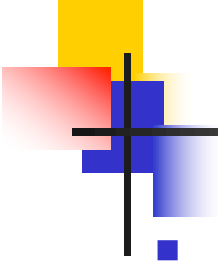
↓

```
{  
  fieldErrors: [  
    {  
      fieldName: "postalCode",  
      message: "size must be between 1 and 20"  
    }  
  ]  
}
```



Capa Acceso a Servicios – diseño

- Dentro de **src/backend** existe un fichero **.js** por cada controlador REST
 - Define tantas funciones como casos de uso expone el controlador REST
 - Oculta los detalles de implementación
 - **catalogService.js**
 - **shoppingService.js**
 - **userService.js**



Capa Acceso a Servicios – diseño

■ catalogService.js

...

```
export const findProducts = async ({categoryId, keywords, page}) => {
```

```
    let path = `/catalog/products?page=${page}`;
```

```
    path += categoryId ? `&categoryId=${categoryId}` : "";
```

```
    path += keywords.length > 0 ?
```

```
        `&keywords=${encodeURIComponent(keywords)}` : "";
```

```
    return await appFetch('GET', path);
```

```
}
```

...

■ shoppingService.js

...

```
export const buy = async (shoppingCartId, postalAddress, postalCode) =>
```

```
    await appFetch('POST', `/shopping/shoppingcarts/${shoppingCartId}/buy`,
```

```
        {postalAddress, postalCode});
```

...

- **src/backend/index.js**

- Estilo similar al de exportación en los módulos

```
...  
import * as userService from './userService';  
import * as catalogService from './catalogService';  
import * as shoppingService from './shoppingService';  
...  
  
export default {..., userService, catalogService, shoppingService};
```

- Estilo similar al de importación de los módulos

```
import backend from '<<...>>/backend';

const example = async () => {
  const shoppingCartId = ...;
  const postalAddress = ...;
  const postalCode = ...;

  const response = await backend.shoppingService.buy(
    shoppingCartId, postalAddress, postalCode);

  if (response.ok) {
    doSomethingWithResult(response.payload);
  } else {
    doSomethingWithErrors(response.payload);
  }
}

example();
```